EVENTFLOW

Large-Scale Event Management Platform

DSA Project Documentation | M1–M6 / CO1–CO6

Submitted by

Team Member	Student ID	Assigned Modules
A.Siva Tejaswi	2500031478	M1 / CO1 and M2 / CO2
P.Saanvi	2500031683	M3 / CO3 and M4 / CO4
S.Vishalini	2500031478	M5 / CO5 and M6 / CO6

Programme & Year:- Btech-PBL & II Year

Section:- 1

Academic Year:- [2026–27]

Faculty / Mentor:- Dr.Maganti Syamala

Department:- CSE

Team Contributions

Member	Student ID	Modules	Contribution
A.Siva Tejaswi	2500031478	M1 / CO1 and M2 / CO2	String algorithms; suffix-array/LCP text analysis; related integration and testing.
P.Saanvi Reddy	2500031683	M3 / CO3 and M4 / CO4	Advanced DP/edit-distance work and network-flow/assignment module.
S.Vishalini	2500031478	M5 / CO5 and M6 / CO6	NP-completeness/approximation and randomized/parallel module.

1. Abstract

EventFlow is a large-scale event-management platform concept designed to organize events, participants, registrations, venues, schedules, staff and volunteers. It applies Data Structures and Algorithms (DSA) to event searching, text indexing, similarity checking, scheduling constraints, resource allocation and analytics.

The currently confirmed Java implementation includes string-matching algorithms, suffix-array/LCP-based text analysis and edit-distance functionality. Network-flow, NP-completeness/approximation, and randomized/parallel algorithms are documented as planned extensions unless their working implementations are present in the submitted source.

2. Problem Statement

Managing many events can make it difficult to find information, detect keywords, compare descriptions, handle misspelled names, and organize schedules and resources. EventFlow demonstrates how suitable algorithms can make these operations more systematic and efficient.

The project uses an event-management scenario to connect DSA concepts with practical operations such as searching event names, finding common text, comparing user input, analyzing capacity and assigning volunteers.

3. Objectives

- Organize events, participants, registrations, venues, schedules and resources.
- Search event/activity names and codes using string-matching techniques.
- Detect multiple event-related keywords in text.
- Index event descriptions and identify shared substrings.
- Compare entered text with stored text using edit distance.
- Establish a roadmap for scheduling, resource allocation, constraints and analytics.

4. Current Implementation Scope

Available project context confirms a Java console-based application with algorithm packages and datasets. Confirmed functionality includes KMP, Z-Function, Rabin–Karp, Aho–Corasick, Suffix Array with LCP/Kasai analysis, and edit-distance functionality. The described flow is Main → UserPortal → EventService → algorithms; keyword detection and description comparison are integrated through the user portal.

SA-IS, suffix trees/suffix automata, Damerau–Levenshtein, Bitmask DP, Matrix-Chain Multiplication and Optimal Binary Search Tree should only be labeled as implemented if working source code and tests are included.CO4: Uses network flow and matching algorithms to optimize volunteer assignments, resource allocation, and capacity management.CO5: Applies NP-completeness and approximation algorithms to analyze scheduling constraints and resolve event conflicts.CO6: Uses randomized and parallel algorithms for event ranking, registration sampling, and efficient analytics.

5. System Architecture

1. User interface: event discovery, details and search (console interface in the Java implementation; separate web frontend prototype where included).

2. Application/service layer: receives user actions and routes requests to event services.

3. DSA layer: performs string searching, keyword detection, text indexing and similarity checks.

4. Data layer: sample datasets/files used by the prototype; persistent database integration is a possible enhancement.

5. Future modules: network-flow analysis, constraint modeling/approximation, and randomized/parallel processing.

6. M1 / CO1 — String Algorithms

KMP (Knuth–Morris–Pratt): searches for an event name, registration code or activity name without repeatedly comparing already matched characters.

Z-Function: computes prefix-match lengths and can help analyze repeated prefixes or text patterns in event descriptions.

Rabin–Karp: uses rolling hashes to search for patterns such as booking or participant codes. Hash matches should be verified to handle collisions.

Aho–Corasick: searches for multiple keywords in one pass, such as “workshop”, “registration”, “AI” or “volunteer”.

Expected outputs include matching positions, detected keywords, and match/no-match results. Suggested tests: pattern present, pattern absent, repeated characters, multiple keywords and empty input.

7. M2 / CO2 — Suffix Structures and Document Similarity

Suffix Array: stores suffix starting positions in sorted order and supports substring-search operations.

LCP/Kasai: calculates longest common prefixes between neighboring suffixes in suffix-array order and helps identify shared text.

SA-IS: a linear-time suffix-array construction approach for large text; list as implemented only if the submitted source contains it.

Suffix Trees/Suffix Automata: alternative structures for substring queries and repeated-substring analysis; include as completed only when code and tests exist.

Suggested tests include short strings, repeated text, shared prefixes and queries with no match.

8. M3 / CO3 — Advanced Dynamic Programming

Levenshtein Edit Distance: calculates the minimum insertions, deletions and substitutions needed to transform one string into another. It can support spelling suggestions.

Damerau–Levenshtein: additionally accounts for adjacent-character transpositions; include only if implemented.

Bitmask DP: a possible approach to selecting compatible activity combinations when the number of activities is limited.

Matrix-Chain Multiplication: finds an efficient parenthesization for a chain of matrix operations.

Optimal Binary Search Tree: builds a search tree intended to reduce expected search cost when access frequencies are known.

Implementation note: edit-distance functionality is confirmed in the available project context, but all the other DP algorithms above are not confirmed as implemented. Treat unverified items as planned.

9. M4 / CO4 — Network Flow

Ford–Fulkerson: computes maximum flow in a capacity-constrained network.

Edmonds–Karp: uses breadth-first search to find augmenting paths and provides a polynomial-time maximum-flow approach.

Dinic's algorithm: uses level graphs and blocking flows for larger flow networks.

Bipartite Matching: assigns eligible staff or volunteers to compatible activities.

König's theorem: relates maximum matching and minimum vertex cover in bipartite graphs, helping analyze assignment conflicts.

EventFlow application: represent facilities/routes as a flow network and volunteers/activities as a bipartite graph. Mark this module as planned unless working code and tests are included.

10. M5 / CO5 — NP-Completeness and Approximation

SAT/3-SAT: represents scheduling rules as Boolean constraints.

3-SAT → CLIQUE: transforms a 3-SAT instance into a graph problem where a clique can represent a consistent selection of literals.

CLIQUE → INDEPENDENT-SET → VERTEX-COVER: uses standard graph relationships to transform related decision/optimization problems.

Vertex-Cover 2-approximation: repeatedly chooses an uncovered edge and adds both endpoints, yielding a cover within a factor of two of optimal for the standard unweighted problem.

EventFlow application: model conflicting activities, venue/time restrictions and required resources. Treat this module as planned unless implementation and validation are present.

11. M6 / CO6 — Randomized and Parallel Algorithms

Randomized QuickSort: sorts events or participants using randomized pivot selection; expected O(n log n) time under standard assumptions.

Reservoir Sampling: maintains a uniform sample from a stream when the total number of registrations is not known in advance.

Miller–Rabin: probabilistic primality test for large integers; its relevance should be linked to a concrete numerical-processing requirement if used.

Blelloch Scan: computes parallel prefix sums, such as cumulative registrations by time slot.

Parallel Reduce: combines partial counts/statistics into totals.

Brent's theorem: provides a theoretical bound relating total work and critical-path depth to parallel execution time.

EventFlow application: streaming samples, cumulative counts and summary statistics. Treat this module as planned unless implementation and performance tests are present.

12. Testing and Validation Plan

String search: test present/absent patterns, patterns at text boundaries, repeated characters and multiple keyword matches.

Suffix structures: test short/repeated strings, common prefixes and substring queries.

Edit distance: test identical strings (distance 0), one-character edits, insertions/deletions and empty strings.

User flow: verify menu options call the expected service and display useful results.

Future graph modules: test capacity limits, disconnected networks, feasible/infeasible assignments and known small graph examples.

Future randomized/parallel modules: compare results with a baseline and verify correctness before measuring performance.

13. Future Enhancements

Integrate the web frontend with Java service endpoints.

Add secure admin and regular-user authentication with role-based access.

Add persistent storage for events, users, registrations, venues and schedules.

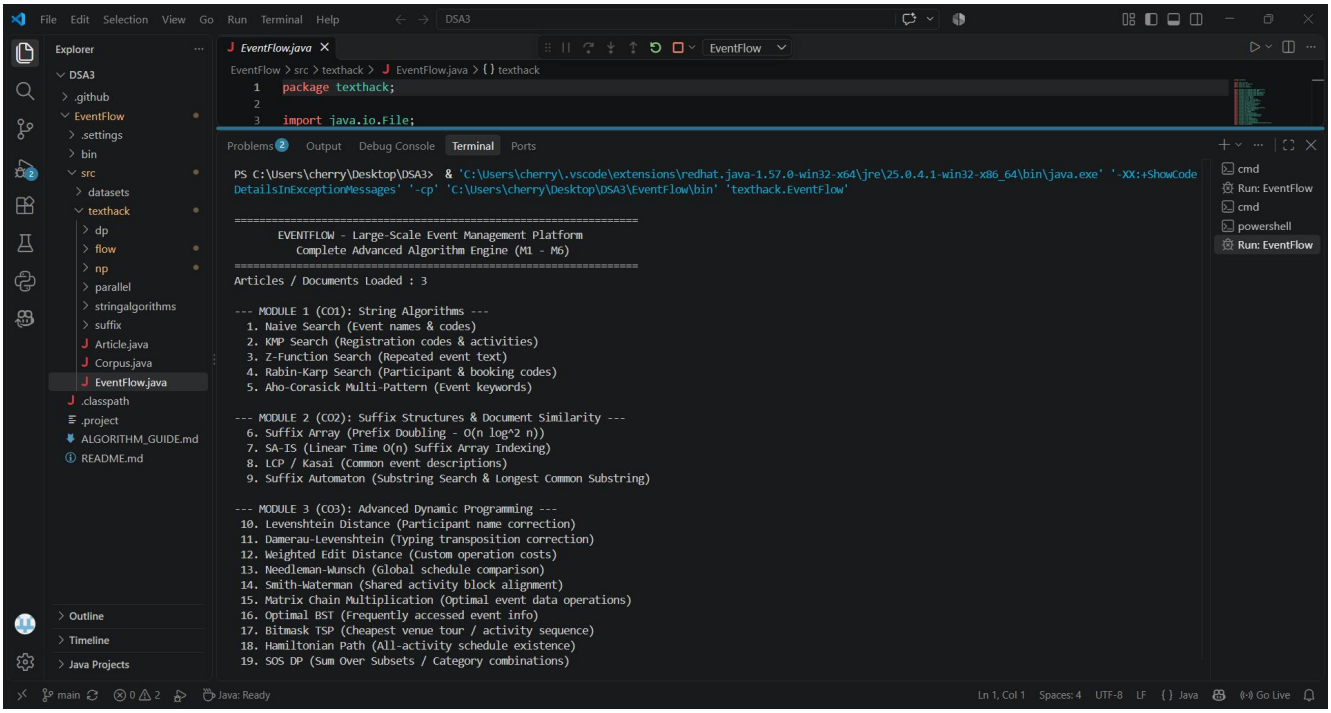
Implement and validate network-flow, approximation, randomized and parallel modules.

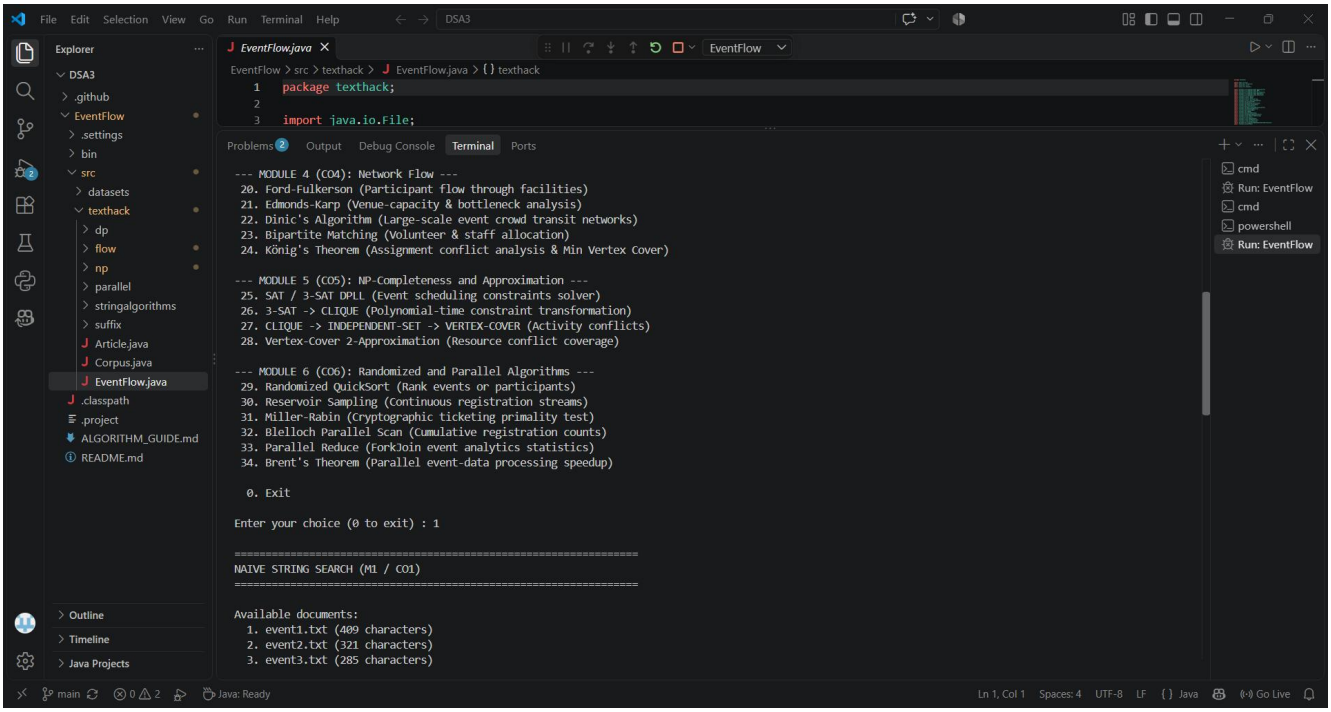
Add automated tests, error handling, deployment instructions and performance measurements.

14. Conclusion

EventFlow applies DSA concepts to an event-management scenario. The confirmed core demonstrates string matching, text indexing/common-prefix analysis and edit-distance comparison. The remaining modules provide a roadmap for capacity management, conflict analysis and large-scale analytics. The final submission should distinguish tested features from proposed work and include screenshots, sample inputs/outputs and test results.

15. Outputs




File Edit Selection View Go Run Terminal Help
DSA3
Explorer
DSA3
.github
EventFlow
.settings
bin
src
datasets
texthack
dp
flow
np
parallel
stringalgorithms
suffix
Article.java
Corpus.java
EventFlow.java
.classpath
.project
ALGORITHM_GUIDE.md
README.md
Outline
Timeline
Java Projects
EventFlow.java
EventFlow
EventFlow > src > texthack > EventFlow.java > { } texthack
1 package texthack;
2
3 import java.io.File;
Problems 2
Output
Debug Console
Terminal
Ports
cmd
Run: EventFlow
cmd
powershell
Run: EventFlow
--- MODULE 4 (CO4): Network Flow ---
20. Ford-Fulkerson (Participant flow through facilities)
21. Edmonds-Karp (Venue-capacity & bottleneck analysis)
22. Dinic's Algorithm (Large-scale event crowd transit networks)
23. Bipartite Matching (Volunteer & staff allocation)
24. König's Theorem (Assignment conflict analysis & Min Vertex Cover)
--- MODULE 5 (CO5): NP-Completeness and Approximation ---
25. SAT / 3-SAT DPLL (Event scheduling constraints solver)
26. 3-SAT -> CLIQUE (Polynomial-time constraint transformation)
27. CLIQUE -> INDEPENDENT-SET -> VERTEX-COVER (Activity conflicts)
28. Vertex-Cover 2-Approximation (Resource conflict coverage)
--- MODULE 6 (CO6): Randomized and Parallel Algorithms ---
29. Randomized QuickSort (Rank events or participants)
30. Reservoir Sampling (Continuous registration streams)
31. Miller-Rabin (Cryptographic ticketing primality test)
32. Blelloch Parallel Scan (Cumulative registration counts)
33. Parallel Reduce (ForkJoin event analytics statistics)
34. Brent's Theorem (Parallel event-data processing speedup)
0. Exit
Enter your choice (0 to exit) : 1
NAIVE STRING SEARCH (M1 / CO1)
Available documents:
1. event1.txt (409 characters)
2. event2.txt (321 characters)
3. event3.txt (285 characters)
main
Java: Ready
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Java
Go Live